Trabajo práctico 7.1
Grafos

Grafos

1. La siguiente es una implementación simple de un grafo G no dirigido utilizando una lista de adyacencia (diccionario de listas).

```
graph = dict()
graph['1'] = ['2']
graph['2'] = ['1','4']
graph['3'] = ['5','6']
graph['4'] = ['2','5','6']
graph['5'] = ['4','6','3']
graph['6'] = ['6','3','5','4']
```

   a. Si el grafo G anterior es de la forma $G = (V, E)$. Exprese el contenido de V, el conjunto de vértices, por extensión.
   b. Exprese el contenido de E, el conjunto de aristas, por extensión.
   c. Dibuje el grafo G.

2. Sea $G_2 = (V_2, E_2)$ un grafo dirigido tal que

$$V_2 = \{1,2,3,4,5,6\}$$
$$E_2 = \{(1,2), (2,1), (2,4), (3,5), (3,6)\}$$

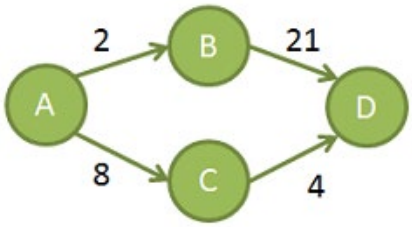
   a. Dibuje G_2
   b. Siguiendo el ejemplo el ejercicio anterior, implemente G_2 utilizando una lista de adyacencia.

3. Implemente la clase Grafo utilizando una lista de adyacencia (un diccionario de diccionarios) con las siguientes operaciones
   a. `__init__(self, nodos=None, aristas=None)`
      el constructor de la clase que opcionalmente acepta una lista de nodos y una lista de aristas
   b. `nuevo_nodo(self, n):`
      que añade un nuevo nodo al conjunto de vértices del grafo
   c. `eliminar_nodo(self, n):`
      que elimina el nodo n en caso de que exista
   d. `conectar(self, n1, n2):`
      que añade una nueva arista entre n1 y n2 si todavía no existe
   e. `desconectar(self, n1, n2):`
      que elimina la arista entre n1 y n2 en caso de que exista
   f. `pertenece(self, n):`
      que retorna True si n es un vértice del grafo. Retorna False en caso contrario
   g. `adyacente(self, n1, n2):`
      que retorna True si n1 y n2 son adyacentes. Retorna False en caso contrario.

4. Implemente una función `hay_camino(G, n1, n2)` que retorne True si existe un camino entre n1 y n2 en G. Y retorne False en caso contrario. Nota: Asuma que G es una instancia de la clase Grafo definida en el ejercicio anterior.

5. Implemente la clase `GrafoM`. La misma debe tener todas las operaciones de la clase Grafo definida anteriormente pero implementadas utilizando una matriz de adyacencia.

6. Implemente una clase `GrafoPonderado` heredera de Grafo que pueda modelar un grafo dirigido con aristas etiquetadas como el de la figura. Asuma que existe a lo sumo una arista entre dos nodos.



7. Implemente una función `camino_posible(G, n1, n2)` que encuentre, si existe, algún camino entre n1 y n2 y devuelva su costo. Si existiera más de un camino se retorna el costo de cualquiera de ellos. En caso de no existir ningún camino, la función debe devolver -1. Asuma que G es una instancia de GrafoPonderado.

8. Implemente una función `mejor_camino(G, n1, n2)` que retorne el camino con menor costo entre n1 y n2 en G. Asuma que G es una instancia de GrafoPonderado.

9. Diseñe una alternativa para poder implementar grafos ponderados (aristas con costo) suponiendo que es posible tener más de una arista entre dos vértices (siempre que cada arista tenga distinto costo).